Geavanceerde computerarchitectuur
Lab sessions

(JLI1QS)

Gheysens Jonathan & Mats Mortier

March 5, 2024



Class: M1EIes

Contents

1 Session 1 - Getting Acquainted with GPU Programming

2 Session 2 - Race conditions

1 Session 1 - Getting Acquainted with GPU Programming

In this lab session, our primary objective was to familiarize ourselves with GPU programming. Unlike CPU programming, which prioritizes sequential thread execution, GPU programming excels in executing multiple threads in parallel. Our focus was on understanding the structure of a program and how it can be optimized for GPU execution.
We began by examining how a program can be broken down into smaller sequential tasks, or threads. To expedite execution, these threads can be run in parallel on the GPU. Threads are organized into blocks, with each block being processed by a distinct GPU core. In this session, our task was to reverse the order of elements in an array.
Initially, we attempted this using only one block containing nine threads. With this setup, all threads had to execute sequentially, leading to nine execution cycles. Next, we divided the threads into three blocks, each containing three threads. This allowed three threads to execute simultaneously, reducing the total execution cycles to three. This can be seen in figure 1.
Finally, we further divided the threads into nine blocks, with one thread per block. This approach necessitated only a single execution cycle, resulting in significant time savings. Although the impact was minimal in this particular case, the advantage would become more pronounced with a substantial increase in the number of threads to be executed.

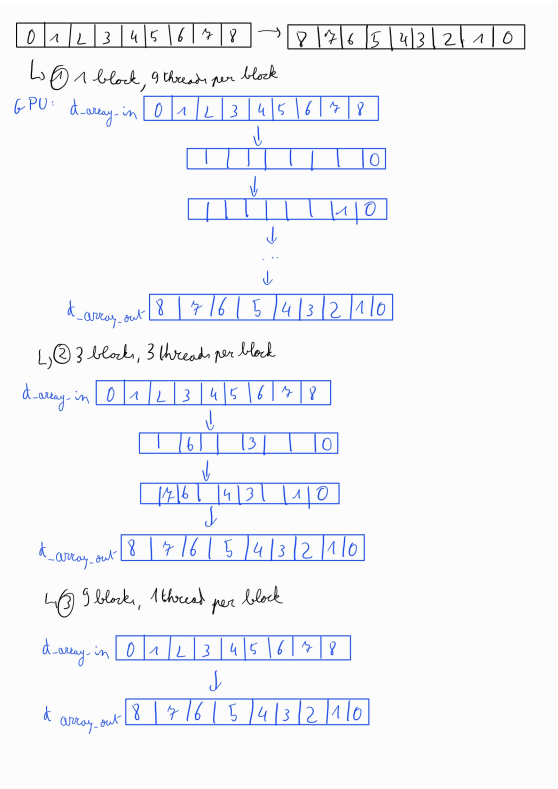


Figure 1: Illustration of blocks and threads

2 Session 2 - Race conditions

During the second lab session we had a look on race conditions and how to avoid them when GPU programming. We did a max detection of the elements of an array using three different methods. The race conditions in this assignment can occur when multiple threads attempt to access the array elements, but the threads of the previous step aren't finished yet. This can be illustrated by figure 3. When one of the threads in the first row hasn't finished yet and the threads of the second step would already start executing, this could give incorrect results. So after each row in figure 3 synchronization must be done to avoid this.
Fist we used a nested for loop on the CPU. For this method we used an extra variable where we stored the maximum value. We initialised this by setting this to the value of the first element of this array. Then using the for loop, we checked if each element of the array greater is then the maximum value variable. If so, we updated the maximum value to the value of the array element value. With this case no race conditions occur.
Secondly, we used a GPU kernel using the atomic max operation. An atomic operation gets executed without interruption.Threads get sequential access to a common resource. Because they have sequential access, there can be no interference and thus, as a result, no race conditions can occur. With the atomic max operation we had a time of 439 μs on an array of size 30000. just for comparison, the nested loop had and time of 2051 μs to find the maximum. So the atomic operations on a large array on the GPU are faster than a nested loop on a large array on the CPU.
Finally, we also used a GPU kernel using reduction. How we will exactly do this can be seen in figure 3, but instead of taking the sum of the elements, we take the maximum value of the two elements. More precisely, we loop over the array with a step size of 1 and search for the maximum value of two elements. The maximum value is placed in source 1 (see figure 2). Before we can continue and loop over the array with a greater step size, we have to synchronize threads first. Otherwise, some threads of the steps could not be finished yet and so we wouldn't get the correct the result. This is what we call race conditions and should be avoided. After the synchronisation we continue looping over the array, but each time with a greater step size as seen in figures 3 and 2. We stop looping when the step size is smaller then log_2(array_size).

Q: Is the result correct?
To fill the array, a loop going from 0 to aray size minus 1 was used each time. With nested loop and the atomic max, array of size 30 000 was used and a maximum was obtained of 29999. This is correct because the largest value contained in the array was 300000 - 1 = 29999.
With the GPU kernel, we had an array of size 2048. From this we got a maximum of 2047, this is correct as well!
Q: How does time increase as a function of input size?
With a very small array we saw that the CPU was always faster than the GPU, but with a huge array the GPU was much faster than the CPU. We can conclude from this that for small array operations the CPU is better but from that working with large arrays we better switch to the GPU. This can be declared by having a look at how CPU and GPU programming works, but this is already explained in section 1 (report of lab session 1).
Q: What happens if the input is larger than 2048 (how would you ensure correct calculation and are there additional synchronization steps)?
The first option is to use an atomic max operation to get the maximum of an array larger than 2048 elements. The atomic max operation is executed on the two maximum elements of each block (original array is splitted in two or more blocks). Secondly, we can also take the maximum of more then two elements within our kernel reduction method. This allows us to use larger

input arrays.

Q: Consider calling the kernel with much less threads than there are data elements. For example, one tenth. How would you ensure all elements get processed?

As mentioned in the answer above , we can split up the original array, so we can process a maximum amount of data elements with the given number of threads. After this first step we can reuse the kernel function to get the maximum element of all the maximum values which were stored in a new array.

Q: How would you tackle a 2D array?

For a 2D array we would take the maximum value of each column and then place them in a new 1D array. Tthis new array can then be used to find the max as in the previous examples for 1D-arrays.

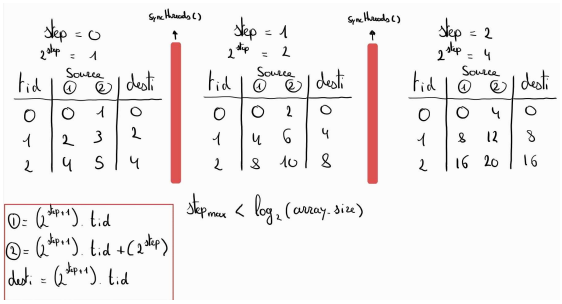


Figure 2: Illustration of reduction-method

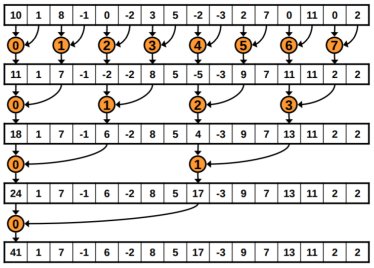


Figure 3: Illustration of array reduction